A Staged Outcome-Blind TMLE Workflow with cleanTMLE

Worked Example

AUTHOR
cleanTMLE Authors

PUBLISHED
May 11, 2026

Read this first

This vignette has two halves. The **first half** teaches the core estimator and shows what a plain end-to-end analysis looks like using `cleanTMLE` as a regular RWE estimation package. The **second half** adds the staged-analysis governance layer (analysis lock, audit log, GO / FLAG / STOP gate) for studies that need a reproducible pre-outcome dossier.

If you are new to targeted-learning methods, read the first half straight through. The vocabulary you need is in the **Concept primer** below. The governance vocabulary (clean room, gate, dossier) does not appear until §"The staged workflow" — by then you will have already seen Table 1, the propensity-score overlap, and a risk-curve plot, so the governance ideas have something concrete to attach to.

A clinical anchor

We use a small simulated cohort (`sim_func1()`) that we will treat as if it came from a pharmaco-epidemiology study. Read it as:

- **Population.** Adult patients enrolled at a tertiary hospital network.
- **Treatment** (`treatment`). A new oral medication started at enrolment versus standard care.
- **Outcome** (`event_24`). A binary event at 24 months (for example, a major cardiovascular event).
- **Baseline covariates** (`age`, `sex`, `biomarker`, `comorbidity`). Standard pre-treatment demographics and risk factors recorded at enrolment.

The numbers are simulated, but the question we are asking is the one a real pharmaco-epi study would ask: **does the new medication change the 24-month event risk, after we adjust for baseline differences between the treated and untreated groups?**

```
set.seed(2026)
dat <- sim_func1(n = 600, seed = 2026)
head(dat[, c("treatment", "event_24", "age", "sex", "biomarker",
             "comorbidity")])
#>   treatment event_24  age sex biomarker comorbidity
#> 1         1        0 55.2   1     0.242           0
#> 2         0        1 39.2   0    -0.420           2
#> 3         0        1 51.4   0    -0.556           0
#> 4         1        0 49.2   0     0.968           0
#> 5         1        1 43.3   1     0.917           0
#> 6         0        0 24.8   1     0.702           1
cat("\nGroup sizes:\n")
```

```
#>
#> Group sizes:
table(treatment = dat$treatment, useNA = "ifany")
#> treatment
#>   0   1
#> 286 314
cat("\nRaw 24-month event rates by treatment arm:\n")
#>
#> Raw 24-month event rates by treatment arm:
round(tapply(dat$event_24, dat$treatment, mean, na.rm = TRUE), 3)
#>     0     1
#> 0.577 0.462
```

Concept primer

Eight terms that appear repeatedly in this vignette and in the targeted-learning literature, in plain language:

- **Estimand.** The exact quantity you want to estimate. For us: the population-average 24-month event-risk difference if everyone took the new medication versus if no one did.
- **Propensity score (PS).** The probability of receiving treatment given a person's baseline covariates. Predicted from a model fit on the data.
- **Overlap (positivity).** Whether treated and untreated groups have similar baseline profiles. Poor overlap means adjustment cannot recover the marginal effect for the full cohort.
- **SMD (standardised mean difference).** A balance metric: the difference in covariate means between treated and untreated, standardised by the pooled SD. Smaller is better; < 0.10 is the convention.
- **ESS (effective sample size, Kish).** The "real" sample size after weighting; a few heavy weights can shrink ESS far below `n`.
- **TMLE (targeted maximum-likelihood estimation).** A four-step recipe: (1) predict the outcome from covariates and treatment; (2) predict the propensity score; (3) *target* the outcome predictions using the propensity score so the resulting treatment-effect estimate has the right calibration; (4) read off the contrast and its standard error.
- **Clever covariate.** The specific function of the propensity score that TMLE uses in step 3. You do not have to compute it by hand; `run_clean_tmle()` does it.
- **EIC (efficient influence curve) variance.** The standard-error formula TMLE uses. Interpretable as a regular Wald-type SE under standard regularity conditions.

Two more terms come up in the second half of the vignette: **plasmode simulation** (a simulation that keeps the real covariate and treatment structure but generates a synthetic outcome with a known truth) and **IPCW** (inverse-probability-of-censoring weighting, used to handle missing outcomes).

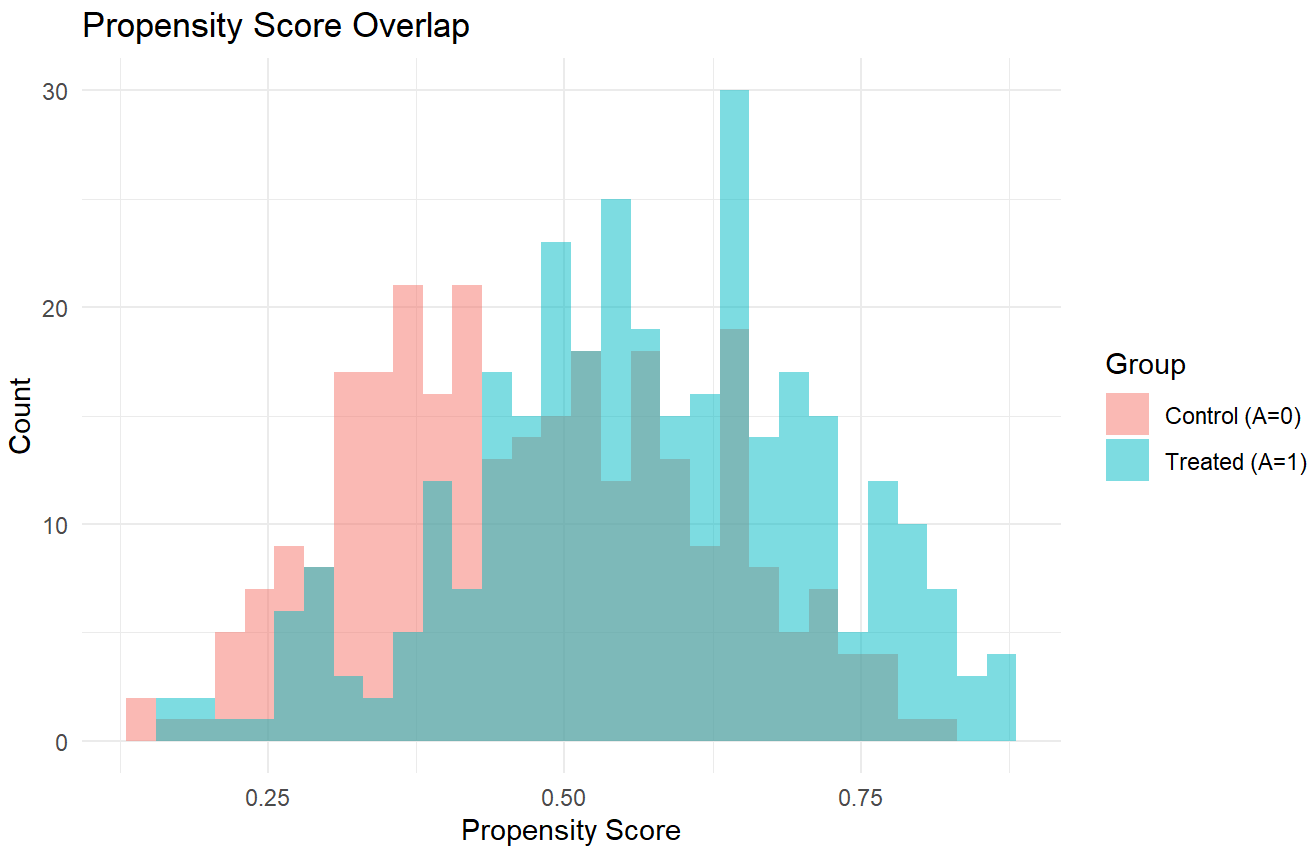
The plain workflow: TMLE in five estimators on one figure

Before introducing any governance machinery, we run the full estimator menu on the clinical anchor and compare them on a single figure. This is the workflow you can lift straight into a methods-development analysis.

```r
# 1. Create a "simple" lock that disables the staged-analysis machinery
#    (no outcome guard, no gate, no required audit). The fingerprint
#    and design diagnostics remain available.
plain_lock <- create_simple_lock(
  data       = dat,
  treatment  = "treatment",
  outcome    = "event_24",
  covariates = c("age", "sex", "biomarker", "comorbidity"),
  sl_library = c("SL.glm", "SL.glmnet", "SL.mean"),
  seed       = 2026
)

# 2. Propensity-score model + overlap and balance diagnostics
ps      <- fit_ps_superlearner(plain_lock)
ps_diag <- compute_ps_diagnostics(ps)
print(ps_diag)
#> Propensity Score Diagnostics
#> ============================
#>
#> Effective Sample Size (Kish ESS):
#>    group   n   ess ess_pct
#>  Treated 314 277.2    88.3
#>  Control 286 256.0    89.5
#>    Total 600 533.3    88.9
#>
#> Standardized Mean Differences:
#>     variable smd_unweighted smd_weighted
#>          age         0.2519       0.0071
#>          sex         0.1824       0.0137
#>    biomarker         0.5815       0.0173
#>  comorbidity        -0.0393      -0.0140
#>
#> Overlap plot available via plot(diag).
```

```r
plot(ps_diag)
```



Propensity-score overlap by treatment arm. Both arms should have density across the same range; gaps signal positivity strain.

The overlap plot is the first thing a reviewer should see. If the two density curves do not cover the same range, adjustment cannot recover a marginal contrast for the full eligible cohort and you will need to consider matching or trimming.

```
# 3. Attach a primary TMLE specification to the lock so the four-step
#    TMLE knows which library and truncation level to use.
primary_spec <- tmle_candidate(
  candidate_id = "plain_tmle",
  label        = "TMLE for the plain workflow",
  q_library    = c("SL.glm", "SL.glmnet"),
  g_library    = c("SL.glm", "SL.glmnet"),
  truncation   = 0.01
)
plain_lock <- lock_primary_tmle_spec(plain_lock, primary_spec)

# 4. Run four estimators on the same lock and same PS
crude  <- run_crude_workflow(plain_lock)
match  <- run_match_workflow(plain_lock, ps_fit = ps)
iptw   <- run_iptw_workflow(plain_lock, ps_fit = ps)

# 5. TMLE via the four-step modular API
g_fit    <- fit_tmle_treatment_mechanism(plain_lock, ps)
Q_fit    <- fit_tmle_outcome_mechanism(plain_lock, g_fit)
```

```
tmle_upd <- run_tmle_targeting_step(g_fit, Q_fit)
tmle     <- extract_tmle_estimate(tmle_upd)
```

```
# 6. Combine the four estimates in one table.
#    Each result has $estimate; CI may be on the top level (run_*_workflow)
#    or under $estimates$ATE (modular TMLE extract).
pick_ci <- function(x) {
  if (!is.null(x$estimates) && !is.null(x$estimates$ATE)) {
    list(est = x$estimates$ATE$estimate,
         lo  = x$estimates$ATE$ci_lower,
         hi  = x$estimates$ATE$ci_upper)
  } else {
    se_or_na <- if (is.null(x$se) || length(x$se) == 0) NA_real_ else x$se
    lo_top   <- if (is.null(x$ci_lower)) x$estimate - 1.96 * se_or_na else x$ci_lower
    hi_top   <- if (is.null(x$ci_upper)) x$estimate + 1.96 * se_or_na else x$ci_upper
    list(est = x$estimate, lo = lo_top, hi = hi_top)
  }
}
plain_results <- data.frame(
  estimator = c("Crude (unadjusted)", "PS matching",
                "IPTW (stabilised)", "TMLE (full cohort)"),
  estimate  = c(pick_ci(crude)$est, pick_ci(match)$est,
                pick_ci(iptw)$est,  pick_ci(tmle)$est),
  ci_lower  = c(pick_ci(crude)$lo,  pick_ci(match)$lo,
                pick_ci(iptw)$lo,   pick_ci(tmle)$lo),
  ci_upper  = c(pick_ci(crude)$hi,  pick_ci(match)$hi,
                pick_ci(iptw)$hi,   pick_ci(tmle)$hi),
  stringsAsFactors = FALSE
)
knitr::kable(plain_results, digits = 4,
             caption = "Four estimators on the plain-mode lock.")
```

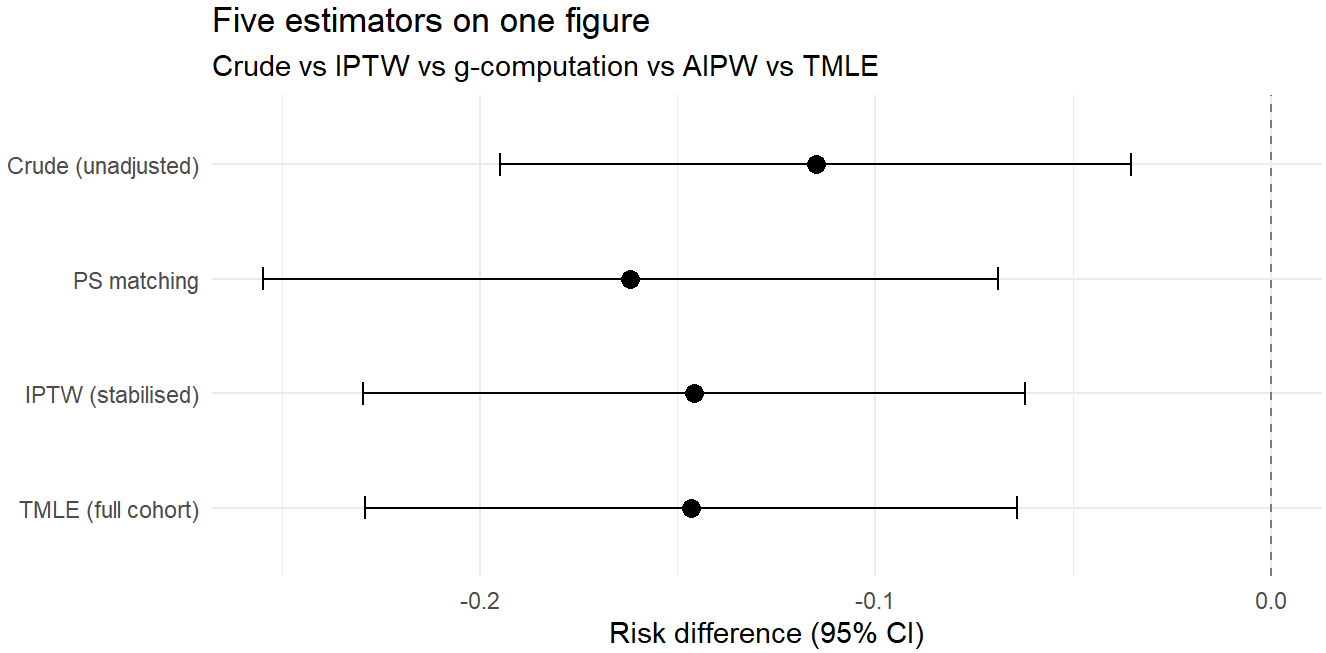
Four estimators on the plain-mode lock.

estimator	estimate	ci_lower	ci_upper
Crude (unadjusted)	-0.1151	-0.1948	-0.0355
PS matching	-0.1619	-0.2548	-0.0691
IPTW (stabilised)	-0.1459	-0.2294	-0.0623
TMLE (full cohort)	-0.1466	-0.2288	-0.0643

(`run_ipcw_tmle()` adds an IPCW sensitivity path for studies with missing outcomes; `estimate_gcomprisk()` and `estimate_aipwrisk()` are also available, but they consume a `specify_models()` object rather than the `cleanroom_lock` shown here. See the function-reference vignette for both.)

```
library(ggplot2)
ggplot(plain_results,
       aes(x = estimate, y = factor(estimator, levels = rev(plain_results$estimator)))) +
```

```
  geom_vline(xintercept = 0, linetype = "dashed", colour = "grey50") +
  geom_point(size = 3) +
  geom_errorbarh(aes(xmin = ci_lower, xmax = ci_upper), height = 0.2) +
  labs(x = "Risk difference (95% CI)", y = NULL,
       title = "Five estimators on one figure",
       subtitle = "Crude vs IPTW vs g-computation vs AIPW vs TMLE") +
  theme_minimal(base_size = 11)
```



Forest plot of the four estimators. The dashed line marks no effect; the adjusted estimators agree closely and shift the crude estimate.

A few things to read from the forest plot:

- The three adjusted estimators (PS matching, IPTW, TMLE) agree closely. When the design diagnostics look reasonable, this is exactly what you should see; substantial disagreement is a signal to slow down.
- The crude (unadjusted) estimate sits noticeably further from the others. The shift between crude and the adjusted estimators is the contribution of the baseline confounders.
- The confidence intervals are not the same width. IPTW's IF-based SE assumes the propensity score is known; in practice it has been estimated, so the IPTW SE is conservative. TMLE's SE accounts for the estimation of the propensity score.
- The matched estimator's CI is wider than the full-cohort TMLE's CI because matching restricts the analysis to the overlap subset and discards unmatched controls.

That is the entire core workflow. Everything in the rest of this vignette is *governance and audit machinery* on top of these five estimators — for studies that need a reproducible pre-outcome dossier — not a different estimator.

When to read the rest

Continue reading if any of the following apply:

- You are running an analysis where the comparative answer will be reviewed by a third party, and you need a reproducible record of every analytic decision made before the comparative answer was seen.
- You want to use the **plasmode candidate-selector** to pick between competing TMLE specifications before unblinding.
- You want to use the **data-quality stress test** to check how the chosen estimator behaves under prespecified threats (measurement error, missingness, unmeasured confounding) before unblinding.
- You are working inside an institutional clean-room arrangement and need the software-side artefacts (lock fingerprint, audit log, decision log, GO / FLAG / STOP record).

If none of these apply, you can stop here and use the plain workflow above. cleanTMLE was designed to be useful at both levels of formality.

Purpose of this vignette

This vignette demonstrates a staged workflow for an observational comparative-effectiveness analysis on a simulated dataset. Design and estimator-selection decisions are made before the observed treatment-outcome association is examined; the goal is reproducible prespecification and a structured documentation of analytic decisions that an external reviewer can follow.

In this manuscript and vignette, *outcome-blind* means that decisions about cohort construction, candidate estimators, simulation scenarios, decision thresholds, and primary-estimator selection are made without inspecting the observed treatment-outcome association. Synthetic outcomes generated for plasmode simulations may be used; the marginal mean of the observed outcome on covariates is also used internally to fit the plasmode generator. The *pre-outcome decision point* is the juncture at which the prespecified diagnostics are reviewed and the analysis is classified GO, FLAG (proceed only with documented review), or STOP before the observed outcome is unmasked for the comparative analysis.

cleanTMLE is not a substitute for target-trial design, source-data validation, phenotype validation, independent oversight, or subject-matter assessment of identification assumptions. The worked example below shows reproducible prespecification, not a regulatory submission template.

Conceptual workflow

The vignette proceeds in the following conceptual steps:

1. **Stage 0 (precondition)**: feasibility assessment, target-trial specification, and protocol / SAP drafting. This is largely external work conducted before any cleanTMLE code is run.
2. **Stage 1a**: lock the estimand, covariates, learners, candidate grid, and decision thresholds.
3. **Stage 1b**: assess cohort adequacy and marginal event support.
4. **Stage 2a**: propensity score, balance, overlap, ESS, and design diagnostics.
5. **Stage 2b**: baseline plasmode candidate selection.
6. **Stage 2c**: plasmode data-quality (DQ) stress testing as a quantitative pre-outcome supplement to the fit-for-purpose review.
7. **Stage 3 (optional)**: negative-control outcome checks.

8. **Pre-outcome decision**: GO / FLAG / STOP.
9. **Stage 4**: authorized primary outcome analysis.
10. Post-outcome sensitivity analyses, interpretation, and archived analysis record.

The Muntner et al. (2020) clean-room governance framework (role separation, restricted access, protocols, audit) is the source of the staging idea and is the broader notion of a clean room. cleanTMLE is the *software layer* of that workflow: it records the analysis lock, runs the design diagnostics and the plasmode evaluation, executes the DQ stress test, applies the pre-outcome decision rule, and writes the audit trail. It does not by itself implement clean-room personnel governance. The canonical stage labels used throughout this vignette are:

```
Stage 0    Feasibility, target-trial specification, protocol / SAP drafting
Stage 1a   Lock estimand, covariates, learners, candidate grid, thresholds
Stage 1b   Cohort adequacy and marginal event support
Stage 2a   PS, balance, overlap, ESS, design diagnostics
Stage 2b   Baseline plasmode candidate selection
Stage 2c   DQ stress testing
Stage 3    Optional negative-control outcome checks
Pre-outcome decision: GO / FLAG / STOP
Stage 4    Authorized primary outcome analysis
Post-outcome: sensitivity analyses and interpretation
```

We use *proceed*, *proceed-with-qualification*, and *do-not-run* in prose; the software shorthand for the same labels is GO, FLAG, and STOP.

> **Warning**
>
> This vignette demonstrates the package workflow. It does not establish exchangeability, measurement validity, phenotype validity, linkage accuracy, or regulatory acceptability.

> **Warning**
>
> Low-replicate examples in this vignette are for workflow demonstration only. Use `n_reps >= 200` before interpreting bias, coverage, or RMSE inferentially.

Pre-outcome study dossier

The reviewer-facing artefact produced by this workflow is a pre-outcome study dossier. It is assembled across the stages demonstrated below, and the review team reads it before authorising the primary analysis. Its components are:

- Protocol and target-trial timing (`attach_estimand()`); see *Define the causal question and estimand*.
- Cohort flow and attrition (`attrition_table()`); see *Stage 1b*.
- Event-process classification (`clean_event_process_table()`, `clean_check_event_processes()`); see *From target trial specification to event-process classification*.
- Covariate and missingness summary (`make_table1()` and missingness checks); see *Stage 1a* covariate sanitisation.

- PS overlap and balance (`compute_ps_diagnostics()`, `love_plot()`, `love_plot_threeway()`); see *Stage 2a*.
- Treatment- and IPCW-weight diagnostics (`clean_weight_diagnostics()`, `extreme_weights()`); see *Outcome-blind diagnostics before fitting the final estimator*.
- Event-count and precision adequacy (`checkpoint_cohort_adequacy()`); see *Stage 1b*.
- Baseline plasmode candidate selection (`run_plasmode_feasibility()`, `select_tmle_candidate()`); see *Stage 2b*.
- DQ stress-test results (`run_plasmode_dq_stress()`, `summarize_dq_degradation()`); see *Stage 2c*.
- Negative-control eligibility and attrition (`run_residual_confounding_stage()`); see *Stage 3*.
- GO / FLAG / STOP checkpoint dashboard (`gate_all()`; aggregate `checkpoint_dashboard()` planned); see *Pre-outcome decision*.
- Decision log and audit log export (`export_decision_log()`); see *Audit trail and reproducibility*.
- Authorisation record for primary analysis (`authorize_outcome_analysis()`); see *Pre-outcome decision*.

The dossier is the artefact the reviewer reads. Comparative treatment-outcome estimates are not part of the dossier and are produced only after authorisation.

Why TMLE fits this workflow

TMLE is well suited to an outcome-blind staged workflow for a small number of inter-related reasons:

- The estimand is declared in advance and the estimator is defined as the targeted update of an initial outcome model toward that estimand, so the analysis target is separable from the nuisance fits.
- The nuisance library, the propensity-score truncation, and the targeting step can all be prespecified and recorded in the analysis lock.
- Targeting decouples the bias of the final estimator from the quality of any single nuisance fit, allowing the use of adaptive learners without driving the choice from the observed outcome association.
- The candidate grid is enumerable: different libraries and truncation rules define distinct candidates that can be screened on synthetic outcomes before authorisation.
- Inference is based on the efficient influence function, which provides a transparent variance estimate compatible with the diagnostics computed at earlier stages.

What each diagnostic answers

Diagnostic	Question answered	What it does not answer
PS / overlap / balance / ESS	Empirical design feasibility	Operating characteristics of any specific TMLE candidate
Baseline plasmode	Estimator behaviour under a locked synthetic outcome model	Behaviour under the realised outcome process
DQ stress testing	Stability under prespecified fit-for-purpose threats	True bias magnitude under unobserved threats

Diagnostic	Question answered	What it does not answer
Negative-control outcomes	Residual bias through a different channel	Primary-estimator behaviour under stress
Post-outcome sensitivity	Robustness to deviations after unblinding	Pre-outcome decisions

Stage 0 as precondition

Stage 0 covers the feasibility assessment, target-trial specification, and protocol / SAP drafting that precede any analytic code. These activities are typically conducted outside the software layer, often involving subject-matter experts, data custodians, and study sponsors. cleanTMLE assumes Stage 0 is complete: the causal question, eligible population, treatment strategies, follow-up window, and primary estimand are already specified in a written protocol before Stage 1a begins. The remainder of this vignette demonstrates Stages 1a through 4.

Define the causal question and estimand

Before writing any analysis code we state the causal question in target-trial-emulation style.

Element	Specification
Population	Adults with simulated disease (the `sim_func1()` cohort)
Treatment strategies	Treatment (A = 1) versus control (A = 0), assigned at baseline
Follow-up	24 months from cohort entry
Outcome	Primary event occurring by 24 months (`event_24`)
Causal contrast	Average treatment effect on the risk-difference scale

The causal estimand is $\psi = E[Y^{a=1}] - E[Y^{a=0}]$, the population-level risk difference under two hypothetical interventions. Because we work with observational data, the quantity we can identify from the observed distribution is the *statistical estimand*

$$\psi = E_W\!\bigl[E[Y \mid A=1, W] - E[Y \mid A=0, W]\bigr],$$

which equals the causal parameter under the standard identification assumptions (consistency, exchangeability, positivity). The package estimates this statistical estimand; causal interpretation relies on the investigator's subject-matter defence of those assumptions.

We choose the risk difference as the primary estimand because it has a transparent absolute-scale interpretation, admits doubly-robust and semiparametric-efficient estimation via TMLE, and is directly relevant to clinical decision making.

From target trial specification to event-process classification

Once the target trial and the statistical estimand are stated, post-baseline events should not be assigned a generic role of "censoring." Each event should be mapped to the causal question, to the corresponding ICH E9(R1) intercurrent-event strategy where applicable, and to the operational role it plays in estimation. The package supports this mapping with three structured helpers: `clean_event_process_table()`, `clean_target_population()`, and `clean_missing_data_plan()`.

```
events <- clean_event_process_table(list(
  list(event_name = "Primary event by 24 months",
       event_variable = "event_24",
       role_in_estimand = "outcome event",
       primary_handling = "modelled directly"),
  list(event_name = "Death from other causes",
       event_variable = "death_other",
       role_in_estimand = "competing event",
       ICH_E9R1_strategy = "composite or competing-risk strategy",
       primary_handling = "competing-risk cumulative incidence",
       justification = "Death precludes measurement of the primary event"),
  list(event_name = "Loss to follow-up",
       event_variable = "lost_fu",
       role_in_estimand = "missing outcome process",
       primary_handling = "IPCW",
       identification_assumption = "MAR conditional on baseline covariates"),
  list(event_name = "Treatment switching",
       event_variable = "treat_switch",
       role_in_estimand = "intercurrent event",
       ICH_E9R1_strategy = "treatment-policy strategy",
       primary_handling = "kept under assigned strategy")
), censoring_handling = "IPCW for missing outcome")
print(events)
#> Event-process classification table
#> ==================================
#>                  event_name        role_in_estimand
#>  Primary event by 24 months           outcome event
#>     Death from other causes         competing event
#>           Loss to follow-up missing outcome process
#>         Treatment switching      intercurrent event
#>                     primary_handling                   ICH_E9R1_strategy
#>                    modelled directly                                <NA>
#>  competing-risk cumulative incidence composite or competing-risk strategy
#>                                IPCW                                <NA>
#>         kept under assigned strategy           treatment-policy strategy
```

```
tp <- clean_target_population(
  target_population = "full eligible population",
  reference_group   = "Control",
```

```
  rationale         = "Marginal effect under each treatment strategy",
  implications_for_interpretation = "ATE on the risk-difference scale")
print(tp)
#> Target population declaration
#> ============================
#> Target population: full eligible population
#> Reference group:   Control
#> Rationale:         Marginal effect under each treatment strategy
#> Interpretation:    ATE on the risk-difference scale
```

```
plan <- clean_missing_data_plan(list(
  "baseline covariate missingness" = list(
    timing = "baseline", presumed_mechanism = "MCAR",
    handling_primary = "median imputation",
    nuisance_model_used = "n/a"),
  "outcome missingness" = list(
    timing = "follow-up", presumed_mechanism = "MAR",
    handling_primary = "IPCW-TMLE",
    nuisance_model_used = "SuperLearner response model",
    identification_assumption = "MAR conditional on (A, W)"),
  "censoring or loss to follow-up" = list(
    timing = "follow-up", presumed_mechanism = "administrative",
    handling_primary = "right-censoring at end of follow-up window")))
print(plan)
#> Missing-data plan
#> =================
#>             variable_or_process presumed_mechanism
#>  baseline covariate missingness               MCAR
#>             outcome missingness                MAR
#>  censoring or loss to follow-up     administrative
#>                            handling_primary
#>                           median imputation
#>                                   IPCW-TMLE
#>  right-censoring at end of follow-up window
```

Stage 1a: Record the analysis specification (lock)

The analysis specification records the treatment, outcome, covariates, candidate estimators, learner library, simulation settings, and random seed. Later steps refer back to this object so that the final analysis can be compared with the prespecified plan, and so that an external reviewer can reconcile the recorded inputs with the analyses that were run.

```
library(cleanTMLE)
set.seed(42)

dat <- sim_func1(n = 400, seed = 42)
```

Covariate sanitisation

Before locking the specification, we sanitise covariates to ensure they are safe for SuperLearner: impute missing values, drop zero-variance columns, and fix formula-unsafe names.

```
clean <- sanitize_covariates(dat,
  covariates = c("age", "sex", "biomarker", "comorbidity"),
  verbose = TRUE)
dat <- clean$data
covs <- clean$covariates
```

```
lock <- create_analysis_lock(
  data          = dat,
  treatment     = "treatment",
  outcome       = "event_24",
  covariates    = covs,
  sl_library    = c("SL.glm", "SL.mean"),
  plasmode_reps = 40L,
  seed          = 42L
)
```

> **Optional: outcome masking**
>
> For analyses that require stricter blinding, `mask_outcome()` can replace the real outcome column with `NA` values inside the lock, preventing any code from accessing the outcome before Stage 4. Call `unmask_outcome(lock, original_lock)` when the pre-outcome gates have been passed and the outcome is needed. In this vignette we proceed without masking for clarity, but the functions are available for production workflows.

We enrich the lock with the estimand, sensitivity plan, and a negative control declaration. These metadata do not alter the reproducibility hash (which depends only on the core analytic specification) but provide the structured record that an outcome-blind staged review requires.

```
lock <- attach_estimand(lock,
  description          = "Effect of treatment on 24-month event risk",
  population           = "Adults with simulated disease",
  treatment_strategies = c("Treatment", "Control"),
  outcome_label        = "Primary event by 24 months",
  followup             = "24 months",
  contrast             = "risk_difference",
  statistical_estimand = "E_W[E(Y|A=1,W) - E(Y|A=0,W)] under exchangeability + positivity"
)
```

```
lock <- declare_sensitivity_plan(lock,
  label       = "truncation_sensitivity",
  description = "Re-estimate under alternate PS truncation thresholds",
  settings    = list(truncation = c(0.01, 0.05, 0.10))
)
```

```
lock <- define_negative_control(lock, "nc_outcome",
  description = "Binary outcome driven by covariates only; treatment has no causal effect"
)
```

```
validate_analysis_lock(lock)
print(lock)
#> cleanTMLE Analysis Lock
#> ======================
#> Data:        400 observations, 12 variables
#> Treatment:   treatment
#> Outcome:     event_24
#> Covariates:  age, sex, biomarker, comorbidity
#> SL library:  SL.glm, SL.mean
#> Plasmode:    40 replicates
#> Seed:        42
#> Locked at:   2026-05-11 14:08:07
#> Hash:        38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b
#>
#> Estimand
#> --------
#> Question:     Effect of treatment on 24-month event risk
#> Population:   Adults with simulated disease
#> Contrast:     Treatment vs. Control
#> Outcome:      Primary event by 24 months
#> Follow-up:    24 months
#> Estimand:     risk_difference
#> Statistical:  E_W[E(Y|A=1,W) - E(Y|A=0,W)] under exchangeability + positivity
#>
#> Sensitivity Plans
#> -----------------
#>   - truncation_sensitivity: Re-estimate under alternate PS truncation thresholds
#>
#> Negative Controls
#> -----------------
#>   - nc_outcome (outcome)
```

We initialise the audit log at this point so that every subsequent stage is recorded.

```
audit <- create_audit_log(lock)
audit <- record_stage(audit, "Stage 1a",
                      "Analysis lock created and estimand attached")
```

File-based pipeline support

In production pipelines each stage runs as a separate script. `save_lock()` / `load_lock()` handle serialisation with automatic hash re-validation, eliminating the boilerplate of manual RDS save/load and integrity checks.

```
# Round-trip demonstration: save, load, verify integrity
tmp_lock_path  <- tempfile("lock_",  fileext = ".rds")
tmp_audit_path <- tempfile("audit_", fileext = ".rds")

save_lock(lock, tmp_lock_path)
save_audit(audit, tmp_audit_path)

# Reload --- load_lock() automatically re-validates the hash
lock_reloaded  <- load_lock(tmp_lock_path)
audit_reloaded <- load_audit(tmp_audit_path)
cat("Lock hash match:", identical(lock$lock_hash, lock_reloaded$lock_hash), "\n")
#> Lock hash match: TRUE
cat("Audit entries match:", length(audit$entries) == length(audit_reloaded$entries), "\n")
#> Audit entries match: TRUE

# Clean up temp files
unlink(c(tmp_lock_path, tmp_audit_path))
```

In a real multi-stage pipeline each stage script would save to a shared results directory (e.g., `save_lock(lock, "results/stage1_lock.rds")`) and the next stage script would `load_lock("results/stage1_lock.rds")` at its top.

We also begin populating the decision log, which records the rationale behind key analytic choices.

```
audit <- record_decision_log_entry(audit,
  stage         = "Stage 1a",
  decision_type = "specification",
  description   = "Analysis lock created with GLM + Mean SL library and 40 plasmode reps",
  rationale     = "Minimal library chosen for vignette reproducibility"
)
```

Stage 1b: Cohort adequacy and marginal event support (Check Point 1)

Before any modelling, we confirm that the cohort has adequate size, event counts, and positivity to support the planned analysis. If this checkpoint fails, the analysis stops before any comparative outcome modelling begins.

```
cp1 <- checkpoint_cohort_adequacy(lock,
  min_n_per_arm  = 50,
  min_events     = 30,
  min_prevalence = 0.01
)
print(cp1)
#> === Check Point 1: Cohort Adequacy ===
#> Decision:  GO
```

```
#> Rationale: Cohort size, event counts, and positivity adequate.
#> Lock hash: 38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b
#> Timestamp: 2026-05-11 14:08:07
#>
#> Metrics:
#>              metric    value
#>             N total 400.0000
#>           N treated 226.0000
#>           N control 174.0000
#>        Events total 204.0000
#>      Events treated 112.0000
#>      Events control  92.0000
#>  Outcome prevalence   0.5100
#>        MDD (approx)   0.1412
audit <- record_checkpoint(audit, cp1)
```

The decision is **GO**. With 400 subjects and adequate events in both arms, we proceed.

Design-stage diagnostics

Before moving to propensity-score modelling we run two additional design-stage diagnostics. `estimate_design_precision()` reports the detectable effect size given the locked sample and event counts, optionally compared against a target minimum detectable difference (MDD). `summarize_event_support()` tabulates event counts by treatment arm and covariate strata, flagging any cells with sparse support.

```
dp <- estimate_design_precision(lock, target_mdd = 0.05)
print(dp)
#> === Design-Stage Precision Summary ===
#>  Metric             Value
#>  N total            400.00000
#>  N treated          226.00000
#>  N control          174.00000
#>  Events total       204.00000
#>  Events (treated)   112.00000
#>  Events (control)    92.00000
#>  Overall prevalence   0.51000
#>  SE proxy (RD)        0.05042
#>  95% CI half-width    0.09882
#>  MDD (80% power)      0.14117
#>
#> Target MDD: 0.05000  |  Feasible: NO
```

> **Note**
>
> The MDD report may show "Feasible: NO" with this small simulated sample (n = 400). This is expected — the vignette uses a deliberately small dataset for fast rendering. In practice you would either increase sample size or relax the target MDD. The MDD check is **informational** and does not trigger STOP; the pipeline continues.

```
es <- summarize_event_support(lock)
print(es)
#>         arm   n events event_rate
#> 1 Treatment 226    112     0.4956
#> 2   Control 174     92     0.5287
#> 3     Total 400    204     0.5100
```

Stage 2a: PS, balance, overlap, ESS, design diagnostics (Check Point 2)

Stage 2a estimates the propensity score and evaluates overlap, balance, and effective sample size. The outcome is never accessed at this stage: only covariates (W) and treatment (A) are used.

Propensity score estimation

```
ps_fit <- fit_ps_glm(lock, truncate = 0.01)
print(ps_fit)
#> Propensity Score Fit (GLM)
#> ==================================
#> Treatment:   treatment
#> Covariates:  age, sex, biomarker, comorbidity
#> PS range:    [0.1715, 0.8647]
#> PS mean:      0.5650
```

Using external or parallel PS

If you have PS estimated outside cleanTMLE (e.g., via a parallel SuperLearner cluster), wrap them with `wrap_ps_fit()`:

```
cl <- parallel::makeCluster(2, type = "PSOCK")
ps_fit <- fit_ps_parallel(lock, cluster = cl, truncate = 0.01)
parallel::stopCluster(cl)

# Or bring-your-own PS:
ps_fit <- wrap_ps_fit(lock, ps_scores = my_ps, method = "external_sl")
```

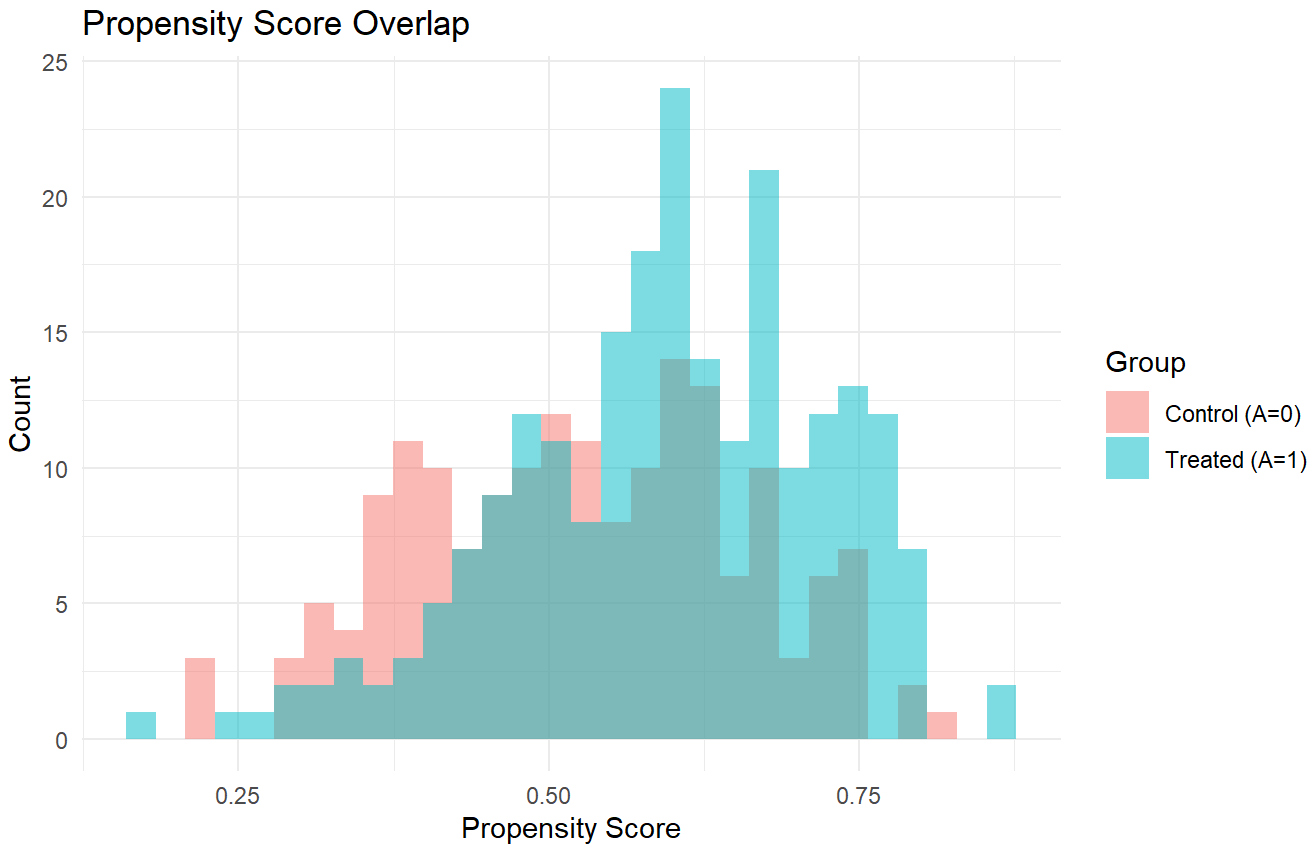
All downstream functions (`compute_ps_diagnostics`, `checkpoint_balance`, `run_iptw_workflow`, etc.) work identically with wrapped PS objects.

Diagnostics

```
diag <- compute_ps_diagnostics(ps_fit)
print(diag)
#> Propensity Score Diagnostics
#> ============================
#>
```

```
#> Effective Sample Size (Kish ESS):
#>    group   n   ess ess_pct
#>  Treated 226 207.4    91.8
#>  Control 174 158.3    91.0
#>    Total 400 365.7    91.4
#>
#> Standardized Mean Differences:
#>     variable smd_unweighted smd_weighted
#>          age         0.1823       0.0182
#>          sex         0.2623      -0.0062
#>    biomarker         0.4237      -0.0229
#>  comorbidity         0.0320       0.0066
#>
#> Overlap plot available via plot(diag).
```

```
plot(diag)
```



Propensity score overlap by treatment arm.

Structured checkpoint

The `checkpoint_balance()` function converts diagnostics into a GO / FLAG / STOP decision based on prespecified thresholds for the maximum weighted SMD and minimum ESS percentage.

```
cp2 <- checkpoint_balance(diag,
  max_smd     = 0.10,
```

```
  min_ess_pct = 50,
  lock_hash   = lock$lock_hash
)
print(cp2)
#> === Check Point 2: Treatment Comparability ===
#> Decision:  GO
#> Rationale: All covariates balanced and ESS adequate.
#> Lock hash: 38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b
#> Timestamp: 2026-05-11 14:08:07
#>
#> Metrics:
#>     variable smd_before smd_after ess_pct
#>          age     0.1823    0.0182      NA
#>          sex     0.2623   -0.0062      NA
#>    biomarker     0.4237   -0.0229      NA
#>  comorbidity     0.0320    0.0066      NA
#>  Total ESS %         NA        NA    91.4
audit <- record_checkpoint(audit, cp2)
```

The decision is **GO**. All covariates are well balanced after weighting, and the effective sample size is adequate.

Stage 2b: Outcome-blind TMLE specification selection

Before accessing the real outcome, we compare *prespecified TMLE specifications* on plasmode-simulated data. Each candidate varies the propensity-score truncation threshold and/or the SuperLearner library used for nuisance estimation. Plasmode simulation preserves the real covariate distribution and treatment mechanism but generates synthetic outcomes with a known true effect, allowing us to evaluate bias, RMSE, and coverage for each candidate specification.

The purpose is *prespecified specification selection*, not performance chasing: the selection rule (here, minimum RMSE) is declared in advance, and the selected TMLE specification is locked into the analysis lock and carried forward exactly to Stage 4 without ad hoc modification.

Note that the PS truncation level is not only a numerical stabilisation parameter: different truncation thresholds imply different effective target populations. Two candidates with different truncation rules are therefore paired estimator-and- estimand objects, and the truncation rule is recorded in the analysis lock alongside the learner library so the estimand-vs-estimator distinction remains auditable.

Estimator selection versus estimand feasibility

A useful mental check before locking the candidate grid:

- If candidates differ only by nuisance learner, screening step, or seed, candidate selection is *estimator* selection. The causal estimand is the same across candidates.
- If candidates differ by PS truncation, overlap restriction, or eligibility filter, they may target *different* effective populations. Selection then has an *estimand* component as well.

In the second case, the prespecified rationale should record the estimand implication for each candidate before any outcome access. If the staged workflow shows that only candidates with severe truncation meet the operating-characteristic thresholds, the practical conclusion is that the original target population may not be supported by the data and the estimand itself needs revision, not just the nuisance fit.

Define TMLE candidates

```
# Define candidate TMLE specifications that vary truncation
candidates <- list(
  tmle_candidate("glm_t01", "GLM, trunc=0.01",
                 g_library = c("SL.glm"), truncation = 0.01),
  tmle_candidate("glm_t05", "GLM, trunc=0.05",
                 g_library = c("SL.glm"), truncation = 0.05),
  tmle_candidate("glm_mean_t01", "GLM+Mean, trunc=0.01",
                 g_library = c("SL.glm", "SL.mean"), truncation = 0.01)
)
```

Plasmode feasibility evaluation

```
plas <- run_plasmode_feasibility(
  lock,
  tmle_candidates = candidates,
  effect_sizes    = c(0.05, 0.10),
  reps            = 40L,
  verbose         = FALSE
)
print(plas)
#> Plasmode-Simulation Feasibility Evaluation
#> ==========================================
#> TMLE candidates: 3
#> Effect sizes evaluated: 0.05, 0.1
#> Replicates per effect size: 40
#>
#> Performance Metrics:
#>  effect_size    candidate    bias    rmse coverage  emp_sd mean_se n_converged
#>         0.05      glm_t01 0.00356 0.04054    1.000 0.04090 0.05149          40
#>         0.05      glm_t05 0.00356 0.04054    1.000 0.04090 0.05149          40
#>         0.05 glm_mean_t01 0.00356 0.04054    1.000 0.04090 0.05149          40
#>         0.10      glm_t01 0.00655 0.04805    0.975 0.04821 0.05107          40
#>         0.10      glm_t05 0.00655 0.04805    0.975 0.04821 0.05107          40
#>         0.10 glm_mean_t01 0.00655 0.04805    0.975 0.04821 0.05107          40
#>  se_cal
#>   1.259
#>   1.259
#>   1.259
#>   1.059
#>   1.059
#>   1.059
```

Select and lock the primary TMLE specification

```
selected <- select_tmle_candidate(plas, rule = "min_rmse")
lock <- lock_primary_tmle_spec(lock, selected)
print(selected)
#> Selected TMLE Candidate: glm_t01
#>   Label:      GLM, trunc=0.01
#>   g-library:  SL.glm
#>   Q-library:  SL.glm
#>   Truncation: 0.01
#>   Rule:       min_rmse
#>   RMSE:       0.04430
#>   Bias:       0.00506
#>   Coverage:   0.988
```

The selected TMLE specification for Stage 4 is **GLM, trunc=0.01** (truncation = 0.01). This specification is now locked into the analysis lock and will be used automatically by `fit_tmle_treatment_mechanism()` and `fit_tmle_outcome_mechanism()`.

Stage 2c: Plasmode data-quality stress test

The DQ stress test is a **quantitative pre-outcome supplement to the fit-for-purpose review**, not a formal quantitative bias analysis (QBA) and not a substitute for source-data validation or post-outcome QBA. The intended sequence is:

1. Fit-for-purpose review identifies plausible threats.
2. Threats are translated into quantitative severity ranges.
3. Severity ranges are locked before primary outcome access.
4. Synthetic outcomes are generated and perturbed.
5. Candidates are evaluated using the same metrics as baseline plasmode selection.
6. The same GO / FLAG / STOP rule is applied.

The DQ stress test documents how candidate estimators behave under the locked threat range; it does not validate the data or replace post-outcome QBA.

Four mechanisms are supported:

1. *Covariate missingness*: a fraction of values is set to `NA` and imputed with the column median.
2. *Treatment misclassification*: an asymmetric pair of (sensitivity, specificity) is applied to A.
3. *Outcome misclassification*: an asymmetric pair of (sensitivity, specificity) is applied to Y, optionally per treatment arm.
4. *Unmeasured confounding*: a latent binary U is sampled and shifts both the treatment mechanism (via a real PS model fitted on the lock data) and the outcome probability by user-specified odds ratios.

```
dq_results <- run_plasmode_dq_stress(
  lock,
```

```
  tmle_candidates = candidates,
  effect_sizes    = c(0.05),
  reps            = 30L,
  data_quality_scenarios = list(
    covariate_missingness = list(
      fractions = c(0.05, 0.10, 0.20)
    ),
    treatment_misclass = list(
      sensitivity = c(0.99, 0.95, 0.90),
      specificity = c(0.99, 0.95, 0.90)
    ),
    outcome_misclass = list(
      sensitivity = c(0.95, 0.90),
      specificity = c(0.99, 0.95)
    ),
    unmeasured_confounding = list(
      U_prevalence   = 0.20,
      U_treatment_OR = c(1.5, 2.0),
      U_outcome_OR   = c(1.5, 2.0)
    )
  ),
  verbose = FALSE
)

deg <- summarize_dq_degradation(dq_results)
head(deg)
```

The *degradation gradient* is the curve of estimator performance against the severity of a prespecified data-quality threat. It is analogous to a dose-response curve for estimator fragility. A shallow gradient suggests a candidate is stable across plausible severities. A cliff suggests that small increases in degradation move performance outside the locked criteria.

`summarize_dq_degradation()` returns a long table comparing each degraded run with the baseline (no degradation): bias change, RMSE ratio, coverage drop. A robust candidate produces a flat curve; a fragile candidate shows a cliff. The table can be filtered to the candidate locked in by `select_tmle_candidate()` and reviewed alongside the SPIFD2 fit-for-purpose narrative before the gate is scored.

The DQ stress test is the package's distinguishing contribution relative to prior outcome-blind plasmode work: standard plasmode selects the candidate with the best baseline RMSE, while `run_plasmode_dq_stress()` allows the analyst to select the candidate that stays inside the gate thresholds across the plausible range of every realistic data threat.

```
audit <- record_stage(audit, "Stage 2b",
  paste("Plasmode evaluation complete; selected TMLE spec:",
        selected$candidate_id))
```

```
audit <- record_decision_log_entry(audit,
  stage         = "Stage 2b",
```

```
  decision_type = "estimator_selection",
  description   = paste("Selected TMLE candidate:", selected$candidate_id),
  rationale     = "Minimum RMSE rule applied to plasmode simulation results"
)
```

Stage 3 (optional): Negative-control outcome checks

A negative-control outcome (NCO) screen is an optional pre-outcome checkpoint that sits between the Stage 2c DQ stress test and the pre-outcome decision. The NCO screen assesses residual confounding using a negative-control outcome — a variable that is known (or strongly believed) to be unaffected by treatment. If the same analytic pipeline detects an association with the negative control, this suggests that confounders remain unbalanced after adjustment.

The negative control outcome `nc_outcome` was generated from covariates only, with no treatment effect, so any estimated association is attributable to residual confounding.

The `run_residual_confounding_stage()` wrapper runs the negative control analysis and generates the checkpoint in a single call, reducing boilerplate and ensuring the two steps stay paired.

```
stage3 <- run_residual_confounding_stage(lock, ps_fit)
print(stage3)
#> === Stage 3: Residual Confounding Assessment ===
#> Negative controls evaluated: 1
#> Flagged:                     0 (0.0%)
#>
#> Summary table:
#>    variable estimate      se p_value flagged failed error
#>  nc_outcome -0.01707 0.04577  0.7092   FALSE  FALSE  <NA>
#>
#> === Check Point 3: Residual Bias ===
#> Decision:  GO
#> Rationale: No negative controls flagged; no evidence of residual confounding.
#> Lock hash: 38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b
#> Timestamp: 2026-05-11 14:08:08
#>
#> Metrics:
#>    variable estimate      se p_value flagged
#>  nc_outcome -0.01707 0.04577  0.7092   FALSE
cp3 <- stage3$checkpoint
audit <- record_checkpoint(audit, cp3)
```

TMLE-based negative controls (doubly robust)

The default `run_negative_control()` uses IPTW. `run_negative_control_tmle()` uses the four-step TMLE pipeline, which is doubly robust and more appropriate for a package built around TMLE.

```
nc_tmle <- run_negative_control_tmle(lock, "nc_outcome", ps_fit)
cat(sprintf("TMLE NC estimate: %.5f  (p = %.4f, method: %s)\n",
            nc_tmle$estimate, nc_tmle$p_value, nc_tmle$method))
#> TMLE NC estimate: -0.01722  (p = 0.7067, method: tmle)
```

The decision is **GO**. No significant association with the negative control was detected, so we proceed to the pre-outcome authorization gate.

Pre-outcome decision

Interpreting GO / FLAG / STOP

The three labels record the result of a prespecified checkpoint, not a judgement about the underlying assumptions:

- **GO**: every prespecified threshold for this checkpoint was met. The workflow may proceed to the next stage.
- **FLAG**: at least one threshold was not met but the result does not block progression. The workflow may proceed under documented review.
- **STOP**: at least one threshold was not met and the checkpoint blocks progression under the locked rule.

An override does not erase the checkpoint result. It records a documented decision to proceed despite the checkpoint result.

The pre-outcome decision summarises whether the prespecified diagnostics support proceeding to outcome analysis, require additional review, or indicate that the study should stop or be redesigned. `authorize_outcome_analysis()` inspects the audit trail and verifies that every required checkpoint has been recorded with a *proceed* (GO) or, when `allow_flag = TRUE`, *pause for review* (FLAG) classification. If any checkpoint is missing or returned *stop* (STOP), the prespecified workflow is not complete and outcome-analysis functions will not run.

`gate_all()` evaluates all checkpoint objects at once and returns the composite classification:

```
composite <- gate_all(cp1, cp2, cp3, allow_flag = TRUE)
print(composite)
#> === Composite Gate ===
#> Decision:  GO
#> Rationale: All checkpoints passed.
#> Timestamp: 2026-05-11 14:08:08
#>
#> Metrics:
#>                                 stage decision
#>          Check Point 1: Cohort Adequacy       GO
#>  Check Point 2: Treatment Comparability       GO
#>            Check Point 3: Residual Bias       GO
```

```
gate <- authorize_outcome_analysis(audit)
print(gate)
#> === Pre-Outcome Gate ===
#> Decision:  GO
#> Rationale: All required stages passed; outcome analysis authorised.
#> Lock hash: 38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b
#> Timestamp: 2026-05-11 14:08:08
#>
#> Metrics:
#>          stage decision_in_audit
#>  Check Point 1                GO
#>  Check Point 2                GO
#>  Check Point 3                GO
audit <- record_checkpoint(audit, gate)
```

```
audit <- record_decision_log_entry(audit,
  stage         = "Pre-outcome decision",
  decision_type = "authorization",
  description   = paste("Outcome analysis classification:", gate$decision),
  rationale     = "All preceding checkpoints passed."
)
```

The pre-outcome classification is **GO**, and outcome-analysis functions may now run on the unmasked data.

What if a checkpoint fails?

In the example above every checkpoint returned GO. In practice, poor overlap or residual confounding can trigger FLAG or STOP decisions. The block below constructs a synthetic failure to show how the pipeline responds:

```
# Simulate a STOP scenario: a checkpoint with very high SMD
fake_diag <- diag
fake_diag$smds$smd_weighted <- c(0.25, 0.30, 0.15, 0.08)  # badly imbalanced
fake_cp2 <- checkpoint_balance(fake_diag, max_smd = 0.10,
                               lock_hash = lock$lock_hash)
cat(sprintf("Fake CP2 decision: %s\n", fake_cp2$decision))
#> Fake CP2 decision: STOP
cat(sprintf("Rationale: %s\n", fake_cp2$rationale))
#> Rationale: Issues: max weighted SMD = 0.300 (> 0.10)

# gate_all() blocks when a STOP is present:
fake_composite <- gate_all(cp1, fake_cp2, cp3, allow_flag = FALSE)
cat(sprintf("\nComposite gate: %s\n", fake_composite$decision))
#>
#> Composite gate: STOP
cat(sprintf("Rationale: %s\n", fake_composite$rationale))
#> Rationale: STOP at: Check Point 2: Treatment Comparability
```

When the gate returns STOP, no outcome code should execute. In `run_clean_tmle()` this is checked automatically; in the modular workflow the analyst inspects the gate decision and branches accordingly.

Outcome-blind diagnostics before fitting the final estimator

Before the outcome is unmasked, we report a compact set of diagnostics that support assessment of identification and estimator stability: treatment overlap, censoring weights (when present), missingness weights (when applicable), weighted SMDs, and effective sample size. These diagnostics should be conducted and the prespecified failure thresholds applied before looking at outcomes when the workflow allows.

```
A <- lock$data[[lock$treatment]]
ps <- ps_fit$ps
w  <- ifelse(A == 1, 1 / ps, 1 / (1 - ps))

W_for_smd <- lock$data[, lock$covariates, drop = FALSE]
weight_diag <- clean_weight_diagnostics(weights = w, treatment = A,
                                        covariates = W_for_smd,
                                        max_weight_threshold = 10,
                                        ess_floor = 0.4 * length(w))
print(weight_diag)
#> Weight diagnostics
#> ==================
#> N = 400
#> Effective sample size (overall): 359.6
#>     1     0
#> 207.4 158.3
#> Extreme weights (> 10.00): 0 (0.0%)
#>
#> Percentiles (overall):
#>    min    p01    p05    p25 median    p75    p95    p99    max   mean
#> 1.1565 1.2657 1.3090 1.5428 1.7962 2.2663 3.3767 4.0602 5.8312 2.0019
#>
#> Standardised mean differences:
#>   smd_unweight smd_weighted
#> 1        0.182        0.018
#> 2        0.262        0.006
#> 3        0.424        0.023
#> 4        0.032        0.007
```

The maximum-weight threshold (10) and the ESS floor (40 % of N) are conservative defaults; the prespecified analysis plan should record values that are appropriate for the study's anticipated overlap, weight distribution, and sample size. When the study uses censoring or missingness weights, `clean_weight_diagnostics()` can be called separately on each weight vector.

Stage 4: Final comparative analysis

Only after all preceding checkpoints pass do we access the real outcome and run the locked analysis.

Unadjusted benchmark

```
crude <- run_crude_workflow(lock)
cat(sprintf("Crude RD: %.4f (95%% CI: %.4f, %.4f)\n",
            crude$estimate, crude$ci_lower, crude$ci_upper))
#> Crude RD: -0.0332 (95% CI: -0.1322, 0.0658)
```

PS matching

```
match_fit <- run_match_workflow(lock, ps_fit)
print(match_fit)
#> Propensity Score Matching Workflow
#> ==================================
#> Matched pairs:   147
#> Unmatched (caliper): 79
#> Risk (treated):  0.4626
#> Risk (control):  0.5306
#> Risk Difference: -0.0680  (95% CI: -0.1778, 0.0417)
#> SE:              0.05601   p-value: 0.2245
```

IPTW

```
iptw_fit <- run_iptw_workflow(lock, ps_fit)
print(iptw_fit)
#> IPTW Workflow
#> =============
#> N:               400
#> Risk (treated):  0.4904
#> Risk (control):  0.5432
#> Risk Difference: -0.0528  (95% CI: -0.1559, 0.0503)
#> SE:              0.05259   p-value: 0.3155
```

TMLE in four steps (primary estimator)

TMLE is the prespecified primary estimator, using the specification selected at Stage 2b and locked into the analysis lock. The locked truncation threshold and learner libraries are applied automatically by the four TMLE functions — no manual specification is needed at this stage.

The package separates the TMLE procedure into four explicit steps aligned with the outcome-blind stages:

1. **Treatment mechanism** (g-model): estimated at Stage 2 without outcome access. The locked truncation is applied automatically.

2. **Outcome mechanism** (\(Q\)-model): requires the real outcome and therefore belongs to Stage 4. The locked Q-library is used.
3. **Targeting step**: fluctuates the initial outcome estimates using the clever covariate \(H(A,W) = A/g(W) - (1-A)/(1-g(W))\).
4. **Estimate extraction**: computes the plug-in ATE, EIC-based SE, and confidence interval.

```
# Step 1: treatment mechanism (re-uses Stage 2 PS)
g_fit <- fit_tmle_treatment_mechanism(lock, ps_fit)
```

```
# Step 2: outcome mechanism (FIRST access to the real outcome)
Q_fit <- fit_tmle_outcome_mechanism(lock, g_fit)
```

```
# Step 3: targeting step
tmle_upd <- run_tmle_targeting_step(g_fit, Q_fit)
```

```
# Step 4: extract final estimate
tmle_fit <- extract_tmle_estimate(tmle_upd)
print(tmle_fit)
#> TMLE Estimate
#> =============
#>
#> ATE:
#>   Estimate: -0.05289
#>   SE:       0.05166
#>   95% CI:   [-0.15415, 0.04836]
#>   p-value:  0.30591
```

```
audit <- record_stage(audit, "Stage 4",
  sprintf("Primary TMLE estimate: %.4f", tmle_fit$estimates$ATE$estimate))
```

Matched-cohort TMLE

`run_matched_tmle()` runs the full TMLE pipeline on a matched subset without creating a separate lock — addressing a common need in pharmacoepidemiology.

```
# Extract matched row indices from the matching workflow.
# run_match_workflow() preserves original rownames in matched_data.
matched_idx <- as.integer(rownames(match_fit$matched_data))
cat(sprintf("Matched observations: %d (of %d total)\n",
            length(matched_idx), nrow(dat)))
#> Matched observations: 294 (of 400 total)

# Run TMLE on just the matched subset
matched_tmle_fit <- run_matched_tmle(lock, ps_fit,
  subset_idx = matched_idx)
print(matched_tmle_fit)
#> TMLE Estimate
```

```
#> =============
#>
#> ATE:
#>   Estimate: -0.07684
#>   SE:       0.06397
#>   95% CI:   [-0.20222, 0.04854]
#>   p-value:  0.22967
```

IPCW-weighted TMLE (missing-outcome sensitivity)

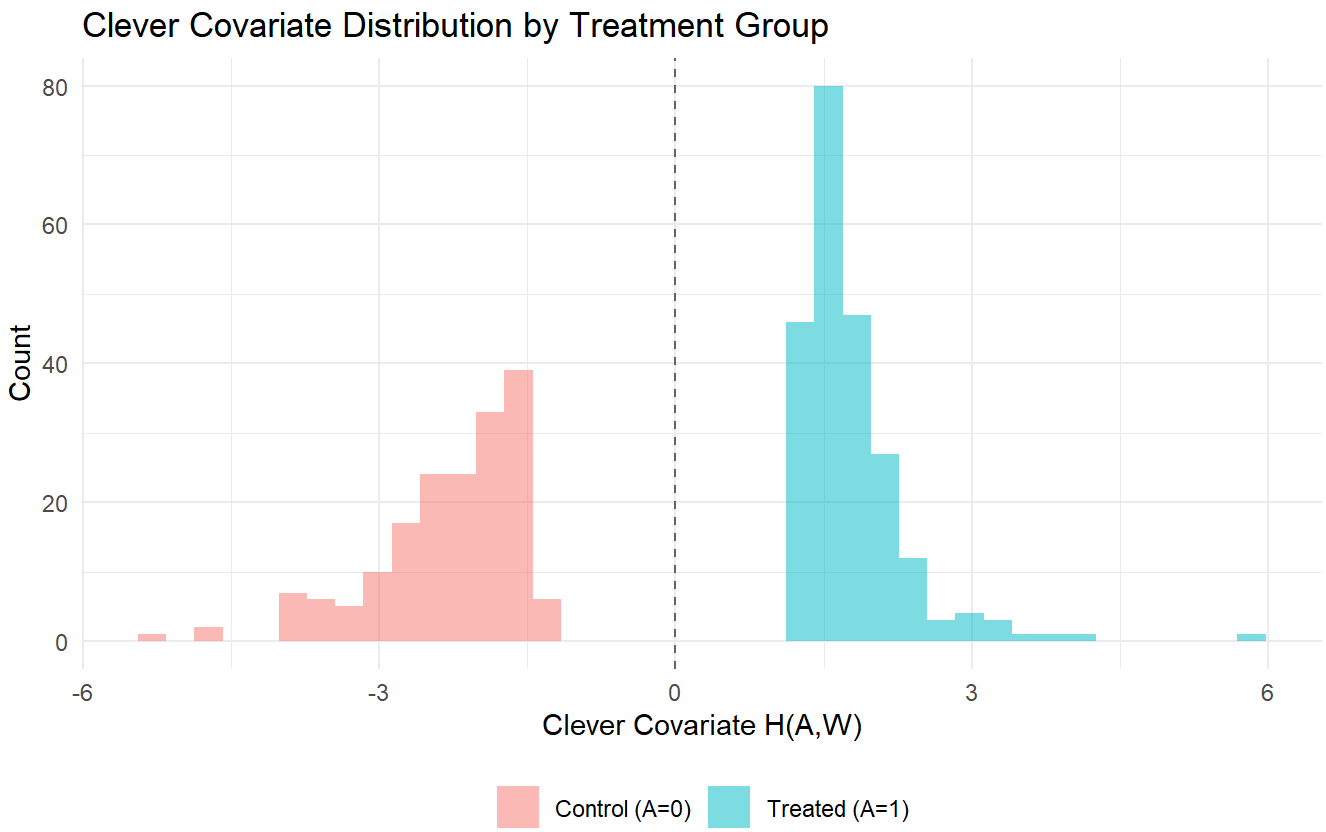
When the outcome is missing for a non-trivial fraction of the analytic cohort and the analyst suspects the missingness mechanism is at-random rather than completely-at-random, `run_ipcw_tmle()` fits a SuperLearner response model \(P(R = 1 \mid A, W)\), builds stabilised inverse-probability-of- censoring weights, and runs TMLE with the censoring weights so the ATE targets the full-cohort marginal estimand rather than the complete-case subset.

```
ipcw_fit <- run_ipcw_tmle(lock, ps_fit)
print(ipcw_fit)
#> TMLE Estimate
#> =============
#>
#> ATE:
#>   Estimate: -0.05010
#>   SE:       0.05267
#>   95% CI:   [-0.15332, 0.05313]
#>   p-value:  0.34150
```

The returned object is a `tmle_fit` and can be combined with the other estimators in [`summarize_cleanroom_results()`] or [`forest_plot()`] for a side-by-side comparison.

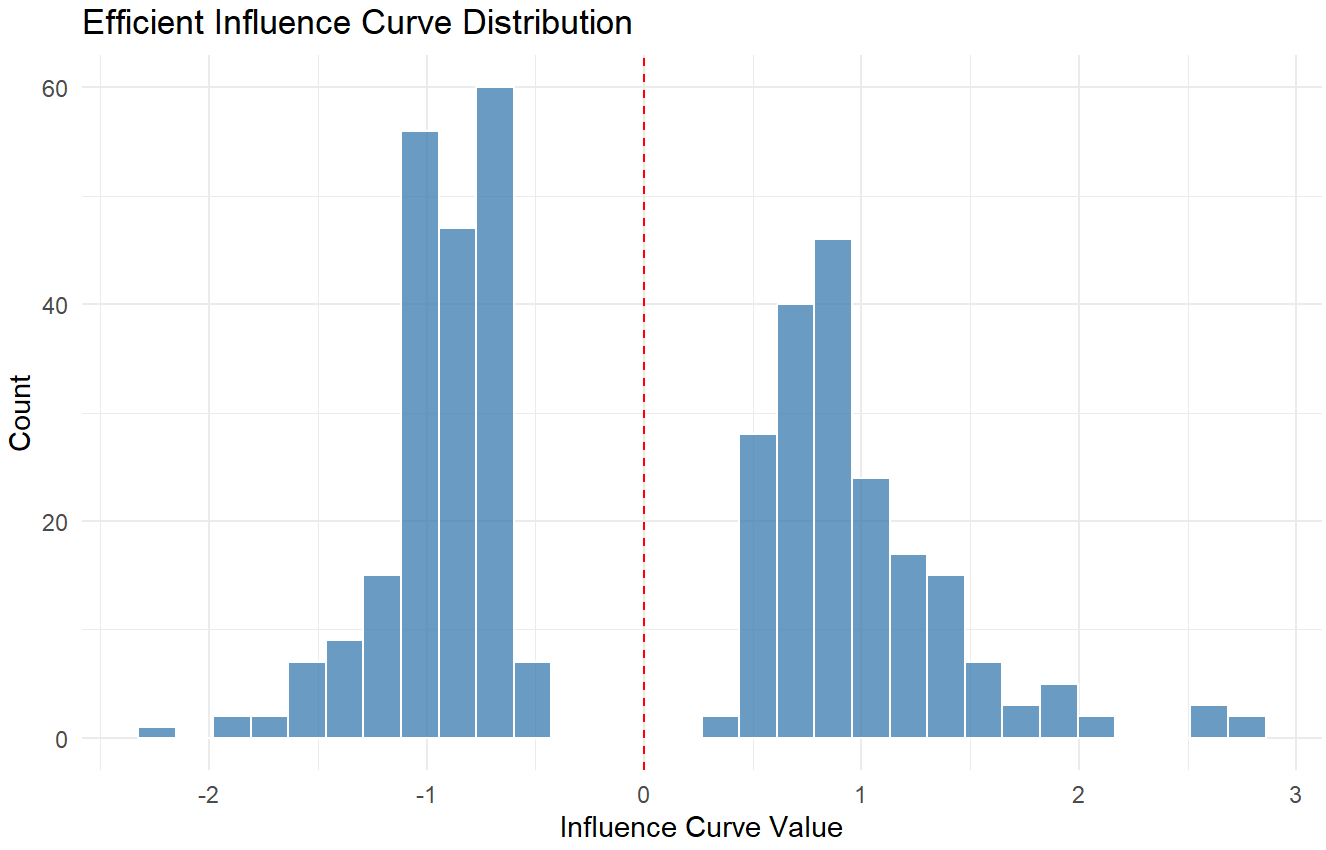
TMLE diagnostics

```
clever_covariate_plot(ps_fit = ps_fit, lock = lock)
```



Clever covariate H(A,W) by treatment group.

```
ic_histogram(tmle_fit)
```

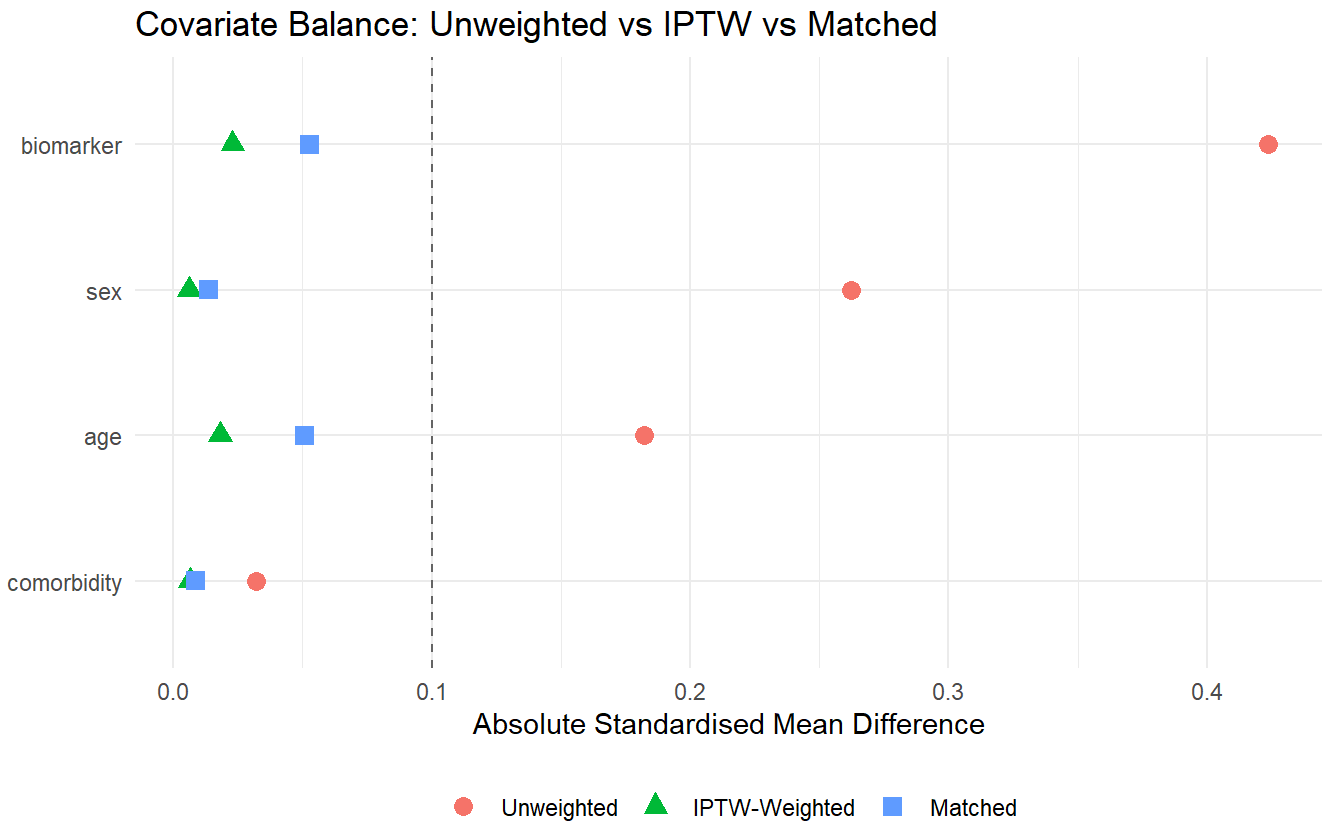


Efficient influence curve distribution.

Three-way balance comparison

`compute_matched_smds()` and `love_plot_threeway()` show covariate balance under three adjustment strategies side by side:

```
m_smds <- compute_matched_smds(dat, "treatment", lock$covariates,
                               subset_idx = matched_idx)
love_plot_threeway(diag, matched_smds = m_smds)
```



Unweighted, IPTW-weighted, and matched SMDs.

Cross-estimator comparison

```
all_fits <- list(
  Crude    = list(estimate = crude$estimate, se = crude$se,
                  ci_lower = crude$ci_lower, ci_upper = crude$ci_upper,
                  p_value  = crude$p_value),
  Matching = match_fit,
  IPTW     = iptw_fit,
  `TMLE (primary)` = tmle_fit
)
summary_tbl <- summarize_cleanroom_results(
  list(Matching = match_fit, IPTW = iptw_fit, `TMLE (primary)` = tmle_fit)
```

```
)
summary_tbl
#>           method    estimate         se   ci_lower   ci_upper   p_value
#> 1       Matching -0.06802721 0.05600620 -0.1777994 0.04174493 0.2245045
#> 2           IPTW -0.05278534 0.05258549 -0.1558529 0.05028221 0.3154747
#> 3 TMLE (primary) -0.05289379 0.05166209 -0.1541515 0.04836391 0.3059102
```

TMLE is the locked primary analysis; matching and IPTW serve as contextual comparators. Agreement across estimators increases confidence that the result is not an artefact of a single modelling strategy.

Reporting cumulative risks and contrasts

Treatment-specific risks at the prespecified follow-up time, the risk difference, the risk ratio, event counts, and the estimator that produced each row are reported in a single structured table via `clean_risk_report_table()`.

```
ate <- tmle_fit$estimates$ATE
crude_r1 <- mean(lock$data[[lock$outcome]][lock$data[[lock$treatment]] == 1])
crude_r0 <- mean(lock$data[[lock$outcome]][lock$data[[lock$treatment]] == 0])

risk_rows <- list(
  list(time_point = 24, treatment_strategy = "Treated",
       n_observed = sum(lock$data[[lock$treatment]] == 1),
       risk = crude_r1 + ate$estimate / 2,
       estimator = "Modular TMLE",
       nuisance_specification = paste(lock$sl_library, collapse = "+")),
  list(time_point = 24, treatment_strategy = "Control",
       n_observed = sum(lock$data[[lock$treatment]] == 0),
       risk = crude_r0 - ate$estimate / 2,
       risk_difference = ate$estimate,
       risk_ci_lower = ate$ci_lower,
       risk_ci_upper = ate$ci_upper,
       estimator = "Modular TMLE",
       nuisance_specification = paste(lock$sl_library, collapse = "+")))
print(clean_risk_report_table(risk_rows))
#> Cumulative-risk reporting table
#> ===============================
#>  time_point treatment_strategy n_observed events      risk risk_ci_lower
#>          24            Treated        226     NA 0.4691283            NA
#>          24            Control        174     NA 0.5551825    -0.1541515
#>  risk_ci_upper risk_difference risk_ratio    estimator
#>             NA              NA         NA Modular TMLE
#>     0.04836391     -0.05289379         NA Modular TMLE
```

Why hazard ratios are secondary in this workflow

cleanTMLE emphasises cumulative-risk contrasts because they map directly to treatment-specific risks under specified strategies and are generally more interpretable for regulatory RWE studies. Hazard ratios may be useful as secondary summaries when proportional hazards is plausible and the estimand is explicitly hazard-based, but they should not replace prespecified cumulative-risk contrasts when the causal question concerns absolute or relative risk by a clinically meaningful follow-up time.

Sensitivity analysis

Truncation sensitivity

The prespecified sensitivity plan varies the PS truncation threshold to assess how much the IPTW estimate depends on extreme weights.

```
sens <- sensitivity_truncation(lock,
  thresholds = c(0.01, 0.025, 0.05, 0.10))
print(sens, row.names = FALSE)
#>  truncation estimate      se ci_lower ci_upper
#>       0.010 -0.05279 0.05259 -0.15585  0.05028
#>       0.025 -0.05279 0.05259 -0.15585  0.05028
#>       0.050 -0.05279 0.05259 -0.15585  0.05028
#>       0.100 -0.05279 0.05259 -0.15585  0.05028
```

E-value

The E-value quantifies the minimum strength of unmeasured confounding (on the risk-ratio scale) needed to explain away the observed association.

```
ate <- tmle_fit$estimates$ATE
# Approximate risk ratio from TMLE risk difference
# Using crude arm risks for illustration
r0_hat <- crude$r0
rr_hat <- (r0_hat + ate$estimate) / r0_hat
rr_lo  <- (r0_hat + ate$ci_lower) / r0_hat
ev <- compute_evalue(rr_hat, ci_bound = min(rr_lo, 1/rr_lo))
cat(sprintf("E-value: %.2f (CI bound: %.2f)\n", ev["e_value"], ev["e_value_ci"]))
#> E-value: 1.46 (CI bound: 2.17)
```

Audit trail and reproducibility

The audit log documents every stage decision, its timestamp, and the lock hash, providing a traceable record of the analytic path.

```
print(audit)
#> cleanTMLE Audit Log
```

```
#> ====================
#> Lock hash:  38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b
#> Entries:    7
#>
#>  stage
#>  Stage 1a
#>  Check Point 1: Cohort Adequacy
#>  Check Point 2: Treatment Comparability
#>  Stage 2b
#>  Check Point 3: Residual Bias
#>  Pre-Outcome Gate
#>  Stage 4
#>  action
#>  Analysis lock created and estimand attached
#>  Checkpoint evaluated: Cohort size, event counts, and positivity adequate.
#>  Checkpoint evaluated: All covariates balanced and ESS adequate.
#>  Plasmode evaluation complete; selected TMLE spec: glm_t01
#>  Checkpoint evaluated: No negative controls flagged; no evidence of residual confounding.
#>  Checkpoint evaluated: All required stages passed; outcome analysis authorised.
#>  Primary TMLE estimate: -0.0529
#>  decision details timestamp
#>                   2026-05-11 14:08:07
#>  GO               2026-05-11 14:08:07
#>  GO               2026-05-11 14:08:07
#>                   2026-05-11 14:08:08
#>  GO               2026-05-11 14:08:08
#>  GO               2026-05-11 14:08:08
#>                   2026-05-11 14:08:08
```

```
build_stage_manifest(audit)
#> Clean-Room Stage Path (lock: 38248c5afde19f901feeb6e19babf86cfe6caa36da118f59bd6d39e7fa146e0b)
#> ------------------------------------------------------------------------------------------
#>   1. Stage 1a: Analysis lock created and estimand attached
#>   2. Check Point 1: Cohort Adequacy: Checkpoint evaluated: Cohort size, event counts, and posi
#>   3. Check Point 2: Treatment Comparability: Checkpoint evaluated: All covariates balanced and
#>   4. Stage 2b: Plasmode evaluation complete; selected TMLE spec: glm_t01
#>   5. Check Point 3: Residual Bias: Checkpoint evaluated: No negative controls flagged; no evid
#>   6. Pre-Outcome Gate: Checkpoint evaluated: All required stages passed; outcome analysis auth
#>   7. Stage 4: Primary TMLE estimate: -0.0529
```

```
trail_df <- export_audit_trail(audit)
knitr::kable(trail_df, caption = "Audit trail for the staged analysis.")
```

Audit trail for the staged analysis.

stage	action	decision	details	timestamp

stage	action	decision	details	timestamp
Stage 1a	Analysis lock created and estimand attached			2026-05-11 14:08:07
Check Point 1: Cohort Adequacy	Checkpoint evaluated: Cohort size, event counts, and positivity adequate.	GO		2026-05-11 14:08:07
Check Point 2: Treatment Comparability	Checkpoint evaluated: All covariates balanced and ESS adequate.	GO		2026-05-11 14:08:07
Stage 2b	Plasmode evaluation complete; selected TMLE spec: glm_t01			2026-05-11 14:08:08
Check Point 3: Residual Bias	Checkpoint evaluated: No negative controls flagged; no evidence of residual confounding.	GO		2026-05-11 14:08:08
Pre-Outcome Gate	Checkpoint evaluated: All required stages passed; outcome analysis authorised.	GO		2026-05-11 14:08:08
Stage 4	Primary TMLE estimate: -0.0529			2026-05-11 14:08:08

Stage path narrative

`summarize_stage_path()` produces a human-readable narrative of the stages traversed, their decisions, and the overall outcome. This is useful for inclusion in study reports or supplementary materials.

```
summarize_stage_path(audit)
#> Stage 1a -> Check Point 1: Cohort Adequacy: GO -> Check Point 2: Treatment Comparability: GO -
```

Decision log

The decision log collects the rationale behind key analytic choices made throughout the staged workflow. `export_decision_log()` returns these entries as a data frame suitable for archival or tabular display.

```
dlog_df <- export_decision_log(audit)
knitr::kable(dlog_df, caption = "Decision log for the staged analysis.")
```

Decision log for the staged analysis.

stage	decision_type	description	rationale	timestamp
Stage 1a	specification	Analysis lock created with GLM + Mean SL library and 40 plasmode reps	Minimal library chosen for vignette reproducibility	2026-05-11 14:08:07
Stage 2b	estimator_selection	Selected TMLE candidate: glm_t01	Minimum RMSE rule applied to plasmode simulation results	2026-05-11 14:08:08
Pre-outcome decision	authorization	Outcome analysis classification: GO	All preceding checkpoints passed.	2026-05-11 14:08:08

A structured decision-log entry recording the Stage 2c outcome and an on-disk export looks like this:

```
audit <- init_decision_log(lock)
audit <- log_decision_entry(audit,
  stage     = "Stage 2c",
  diagnostic = "DQ stress test minimax",
  criterion  = "All candidates within bias/coverage thresholds across severities",
  decision   = "GO",
  rationale  = "Minimax candidate glm_t01 selected; full-cohort plasmode confirms.",
  reviewer   = "lead_analyst",
  y_visible  = FALSE,
  action     = "Lock glm_t01 as primary; record DQ summary in dossier."
)
export_decision_log(audit, "decision_log.csv")
```

Summary

The worked example illustrates the canonical stages:

1. **Stage 0** — feasibility, target-trial specification, and protocol / SAP drafting (precondition, external to the package).
2. **Stage 1a** — lock the estimand, covariates, learners, candidate grid, and decision thresholds.
3. **Stage 1b** — cohort adequacy and marginal event support (Check Point 1, with design-precision diagnostics).
4. **Stage 2a** — propensity score, balance, overlap, ESS, and design diagnostics (Check Point 2).
5. **Stage 2b** — baseline plasmode candidate selection using the prespecified rule.
6. **Stage 2c** — plasmode DQ stress testing on the locked severity ranges from the fit-for-purpose review.
7. **Stage 3 (optional)** — negative-control outcome checks.
8. **Pre-outcome decision** — GO (proceed), FLAG (proceed-with-qualification), or STOP (do-not-run), based on the recorded checkpoint results.
9. **Stage 4** — authorized primary outcome analysis on the unmasked data.
10. **Post-outcome** — sensitivity analyses, interpretation, and archived analysis record (analysis specification, audit trail, and decision log).

The package includes additional functions used along the way: `sanitize_covariates()` for covariate preparation, `run_negative_control_tmle()` for residual-bias screening, the composite `gate_all()` for evaluating checkpoint objects together, `run_matched_tmle()` and the four-step TMLE pipeline for the final analysis, `compute_evalue()` and `sensitivity_truncation()` for sensitivity analyses, and `run_plasmode_dq_stress()` with `summarize_dq_degradation()` for studies that want to combine candidate selection with a quantitative pre-outcome stress test.

Integration with real-world pipelines

The package now supports common production patterns:

- **External PS**: `wrap_ps_fit()` wraps PS from parallel SuperLearner, Bayesian models, or external software into a compatible `ps_fit`.
- **Parallel SuperLearner**: `fit_ps_parallel()` accepts a PSOCK cluster.
- **File-based pipelines**: `save_lock()` / `load_lock()` and `save_audit()` / `load_audit()` handle RDS serialisation with automatic hash re-validation.
- **YAML config**: `create_analysis_lock_from_yaml()` reads analytic specifications from a YAML configuration file.
- **Matched-cohort TMLE**: `run_matched_tmle()` runs TMLE on a subset without creating a separate lock.

The package does not enforce institutional governance (role separation, independent review boards, blinded data managers); those are organisational responsibilities. What it does provide is software scaffolding that makes the staged workflow reproducible, auditable, and less susceptible to unintentional outcome-informed decision making.

```
sessionInfo()
#> R version 4.4.2 (2024-10-31 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#>
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/Los_Angeles
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] ggplot2_4.0.2   nnls_1.6        cleanTMLE_0.1.1
#>
#> loaded via a namespace (and not attached):
#>  [1] sandwich_3.1-1        utf8_1.2.4            generics_0.1.3
#>  [4] shape_1.4.6.1         lattice_0.22-6        digest_0.6.37
#>  [7] magrittr_2.0.3        evaluate_1.0.5        grid_4.4.2
#> [10] RColorBrewer_1.1-3    iterators_1.0.14      fastmap_1.2.0
#> [13] foreach_1.5.2         jsonlite_2.0.0        Matrix_1.7-1
#> [16] glmnet_4.1-8          survival_3.7-0        fansi_1.0.6
#> [19] scales_1.4.0          codetools_0.2-20      cli_3.6.5
#> [22] rlang_1.1.7           splines_4.4.2         withr_3.0.2
#> [25] yaml_2.3.10           tools_4.4.2           dplyr_1.1.4
#> [28] vctrs_0.7.1           R6_2.6.1              zoo_1.8-15
```

```
#> [31] lifecycle_1.0.5      htmlwidgets_1.6.4    SuperLearner_2.0-29
#> [34] pkgconfig_2.0.3      pillar_1.9.0         gtable_0.3.6
#> [37] tmle_2.0.1.1         glue_1.8.0           Rcpp_1.0.13-1
#> [40] xfun_0.49            tibble_3.2.1         tidyselect_1.2.1
#> [43] knitr_1.49           farver_2.1.2         htmltools_0.5.8.1
#> [46] rmarkdown_2.29       labeling_0.4.3       gam_1.22-5
#> [49] compiler_4.4.2       S7_0.2.1             WeightedROC_2020.1.31
```